

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv("/content/drive/MyDrive/studies/#TE/SEM5/Internship/internship/joint_internship/mu_aqi.csv")
```

df

	T	TM	Tm	H	VM	VV	V	PM_2_5	PM10	AQI
0	23.4	30.3	19.0	59	5.4	6.3	4.3	284.795833	64	68
1	22.4	30.3	16.9	57	7.6	6.9	3.3	239.695455	70	77
2	24.0	31.8	16.9	51	5.4	6.9	2.8	182.187500	64	32
3	22.8	30.2	17.6	55	7.6	7.7	3.7	264.064286	93	93
4	23.1	30.6	17.4	46	5.4	6.9	3.3	223.208333	55	60
...	...	...	...	...	...	...	...	...	...	...
125	24.3	30.8	19.2	72	3.5	6.9	0.9	33.166667	337	364
126	23.9	31.4	19.0	76	5.4	6.9	1.5	30.625000	306	350
127	22.9	31.4	19.5	80	11.1	6.9	3.7	30.333333	394	441
128	22.2	27.7	20.0	85	7.6	6.3	4.8	0.000000	223	265
129	21.8	25.4	19.5	89	5.4	5.5	2.8	0.000000	170	275

130 rows × 10 columns

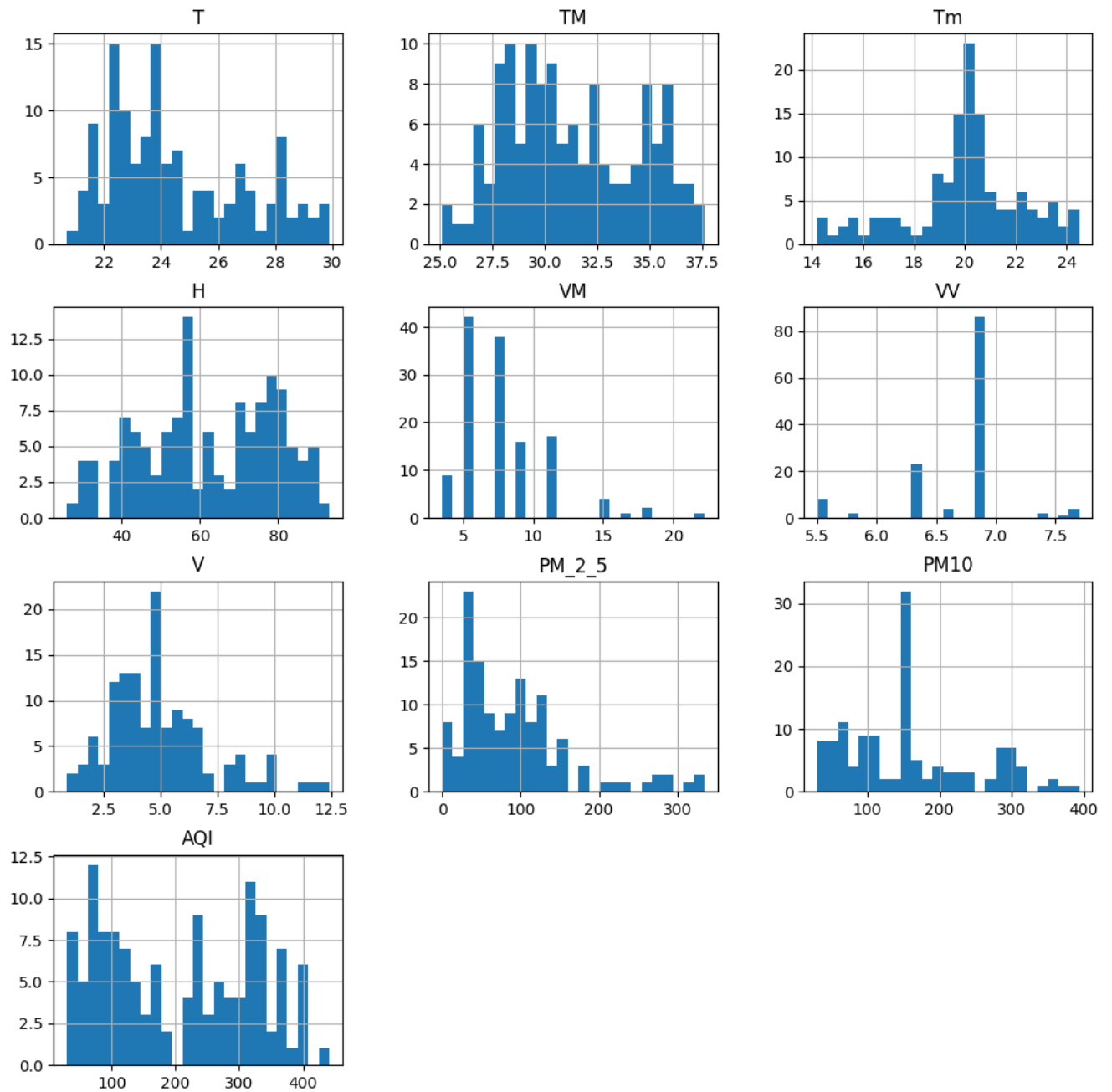
df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 130 entries, 0 to 129
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    T           130 non-null    float64
1    TM          130 non-null    float64
2    Tm          130 non-null    float64
3    H           130 non-null    int64
4    VM          130 non-null    float64
5    VV          130 non-null    float64
6    V           130 non-null    float64
7    PM_2_5      130 non-null    float64
8    PM10        130 non-null    int64
9    AQI         130 non-null    int64
dtypes: float64(7), int64(3)
memory usage: 10.3 KB
```

df.describe()

	T	TM	Tm	H	VM	VV	V	PM_2_5	PM10	AQI
count	130.000000	130.000000	130.000000	130.000000	130.000000	130.000000	130.000000	130.000000	130.000000	130.000000
mean	24.526923	31.219231	20.014615	62.146154	7.851538	6.719231	4.949231	91.678790	159.076923	205.438462
std	2.378840	3.145493	2.237181	17.006662	3.217135	0.449305	2.225617	72.060636	89.507837	117.251205
min	20.700000	25.100000	14.200000	26.000000	3.500000	5.500000	0.900000	0.000000	30.000000	30.000000
25%	22.600000	28.625000	19.200000	50.000000	5.400000	6.375000	3.500000	38.333333	90.250000	94.250000
50%	23.900000	30.600000	20.100000	62.000000	7.600000	6.900000	4.600000	78.750000	150.000000	224.000000
75%	26.475000	34.075000	21.000000	77.000000	9.400000	6.900000	6.050000	122.093750	206.750000	313.500000
max	29.900000	37.600000	24.500000	93.000000	22.200000	7.700000	12.400000	334.341177	394.000000	441.000000

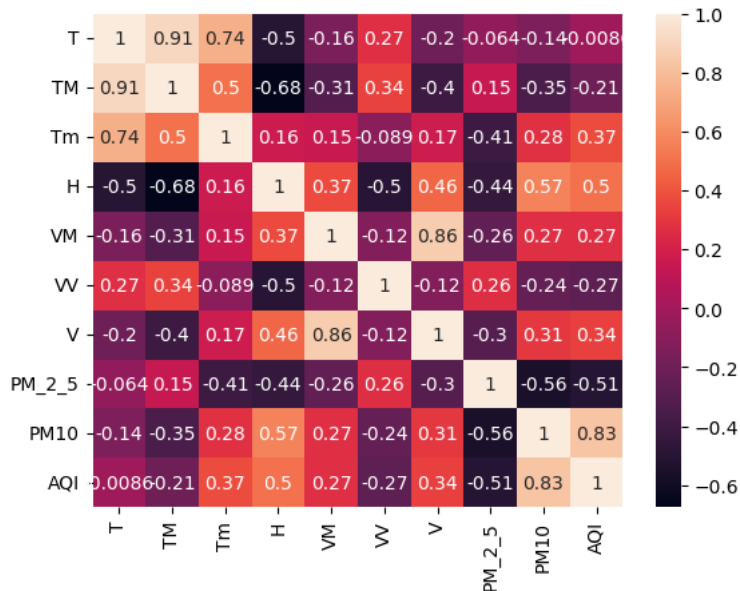
```
# Plotting frequency distribution for each variable
df.hist(figsize = (12,12), bins = 25)
plt.show()
```



```
# Measure the correlation and plot the heatmap
import seaborn as sns
```

```
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot = True)
```

<Axes: >



```
X = df.drop('AQI', axis=1)
y = df['PM10']
```

```
# split the data into train and test
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state=42)
```

```
# Linear Regression
from sklearn.linear_model import LinearRegression
```

```
# Model Object
lr_reg = LinearRegression()
```

```
# Fit the model
lr_reg.fit(X_train, y_train)
```

```
LinearRegression()
```

```
# make predictions and check the error
lr_preds_1 = lr_reg.predict(X_test)
```

```
# Metrics
from sklearn.metrics import mean_squared_error
```

```
mse_1 = mean_squared_error(y_test, lr_preds_1) #-> Compute the mean squared error
rmse_1 = np.sqrt(mse_1) #-> Compute the Root Mean Squared Error
```

```
print('==== PERFROMANCE ====')
print('The mean squared error is {:.3f}'.format(mse_1))
print('The root mean squared error is {:.3f}'.format(rmse_1))
```

```
==== PERFROMANCE ====
The mean squared error is 0.000
The root mean squared error is 0.000
```

```
predicted_dataframe = pd.concat([X_test], axis=1)
predicted_dataframe[' Expected_aqi'] = lr_preds_1
```

```
predicted_dataframe.head()
```

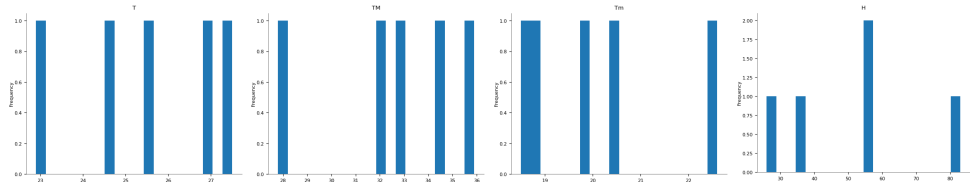
index	T	TM	Tm	H	VM	VV	V	PM_2_5	PM10	Expected_aqi
55	27.5	35.9	22.6	56	11.1	6.3	5.9	104.625	112	111.99999999999996
40	26.9	34.6	19.9	37	5.4	6.9	5.0	98.70833333	150	149.99999999999994
19	24.6	32.2	18.8	55	3.5	6.9	2.2	54.79166667	86	86.0
31	25.5	33.0	18.5	26	5.4	6.9	3.9	36.35714286	102	102.0
115	22.9	27.8	20.4	83	5.4	6.3	4.4	35.29166667	177	177.00000000000003

Show 50 per page

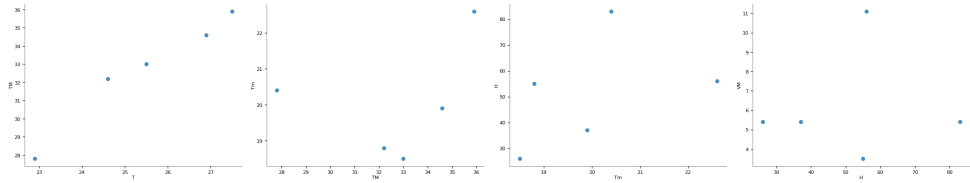


Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Distributions



2-d distributions



Time series

